

MiniBug-RL: Tiny, Reproducible Reinforcement Learning for Verifiable Python Repair

BurnyCoder

<https://github.com/BurnyCoder/llm-rl-software-engineering>

September 4, 2026

Abstract

We ask whether a reinforcement-learning-from-verifiable-rewards loop for single-function software repair can complete on one recorded 8 GB consumer GPU. We train a rank-16 LoRA adapter on the 0.49B-parameter Qwen2.5-Coder model for 100 Group Relative Policy Optimization steps. Four candidate repairs per prompt receive one composite reward: structural rejects are handled on the host, deterministic cache hits reuse an earlier breakdown, and valid cache misses run hidden unit tests in fresh resource-limited Docker containers. Training completed in 1,174.08 seconds with 1.72 GiB peak allocated GPU memory and changed all 336 trainable adapter tensors. Validation improved from 7/12 to 8/12 greedily solved tasks and selected the 100-step checkpoint. On the 12-task internal set held out from model optimization and selection, full greedy solves rose from 5/12 to 6/12, but mean greedy hidden-test correctness changed from 0.6083 to 0.6042 (paired difference -0.0042, paired percentile-bootstrap 95% interval [-0.2917, 0.2625]). On frozen HumanEvalFixDocs-Python, greedy pass@1 changed from 37/164 to 38/164 (difference +0.0061, paired percentile-bootstrap 95% interval [-0.0183, 0.0366]). Invalid internal candidates were 19/60 versus 1/60. The result establishes an end-to-end, publicly reloadable RL pipeline and an observed output-validity difference; it does not establish a statistically conclusive general repair-capability gain.

1 Research question and contribution

Software is unusually suitable for verifiable reinforcement learning because candidate behavior can be checked by executable tests rather than a learned preference model. The practical obstacle is that serious repository-scale agent benchmarks often combine large policies, long contexts, tool orchestration, and substantial compute. This experiment narrows the domain to deterministic single-function Python repair and asks:

Can a sub-billion-parameter code model receive genuine group-relative policy updates from hidden tests, on one 8 GB GPU, while retaining explicit split discipline, resource-limited execution isolation, auditable evidence, and a resulting model that another user can load?

The gate pre-specified in producing source commit 7027bc55baecc00fad51cbe2b8f030dca2c91c1e was that 100 low-rank GRPO steps would improve validation hidden-test correctness by at least 0.05 or solve one additional validation task, without worsening the aggregate format/runtime/policy

status rate [13]. The experiment contributes (i) a version-pinned implementation; (ii) a project-authored 36/12/12 train/validation/test curriculum with structural leakage checks; (iii) a Docker-mediated reward path; (iv) selection-locked internal and external paired evaluation; and (v) public merged and adapter artifacts. Project authorship is a repository declaration, not an independent originality audit.

The scope is intentionally modest. A function-repair policy is not a repository agent, and passing small unit-test suites is not evidence of broad software-engineering autonomy.

2 Foundations and implementation choice

GRPO was introduced as a policy-optimization variant that forms relative advantages from groups of sampled completions while avoiding a learned value model [1]. TRL exposes the algorithm, custom reward functions, PEFT integration, and DR-GRPO loss in a maintained trainer interface [2]. LoRA represents the update with small trainable low-rank matrices while leaving base weights frozen [3, 4]. These properties make the combination suitable for limited memory.

Qwen2.5-Coder provides instruction-tuned code models down to a reported 0.49B parameters [5, 6]. The pinned base is

```
Qwen/Qwen2.5-Coder-0.5B-Instruct
commit ea3f2471cf1b1f0db85067f1ef93848e38e88c25.
```

The pinned public DebugArena model card motivated the particular 0.5B model, four-completion group, and 10^{-5} learning rate [7]. Its illustrative snippet does not define its reward function, dataset construction, adapter attachment, isolation, held-out protocol, or a released trained checkpoint. MiniBug-RL therefore uses the installed, version-pinned TRL, PEFT, Transformers, and Hub APIs rather than treating that snippet as a complete recipe.

3 Method

3.1 Curriculum and prompt contract

The project-authored curriculum contains 60 short, import-free functions: 36 training, 12 validation, and 12 final-test tasks. Families include incorrect comparisons, boundary and off-by-one errors, bad state updates, accumulation errors, indexing mistakes, and Boolean logic. Each task has a natural-language specification, buggy source, two public JSON examples, and no gold repair; 59 tasks have four hidden JSON cases and one has five [14].

The model receives only the specification, buggy function, and public examples. It is instructed to return exactly one complete replacement function. A versioned manifest hashes canonical task records and normalized abstract syntax trees. Bound names are alpha-normalized and literal values are masked by type before structural hashing; no normalized template crosses a split. Every original bug was run in the real container and shown to fail at least one hidden case before training [13, 14].

3.2 Reward and isolation

For a valid candidate with n hidden tests, let p be the fraction passed and I indicate a full solve. The ordinary successful-execution reward is

$$r = p + I[p = 1] + 0.1, \quad (1)$$

so a syntactically valid but wholly wrong function earns 0.1 and a complete repair earns 2.1. Invalid structure returns -0.3 , a prohibited construct returns -1.0 , and a valid function that crashes or times out receives $0.1 - 0.25 = -0.15$. Expected values remain in the trusted host process; only candidate source and JSON call inputs cross the container boundary. Actual results return to the host for comparison [13].

Across the two-step smoke and 100-step main training, the 404 rollout records comprised 38 host-side structural rejections, 175 deterministic cache hits, and 191 valid cache misses. Each valid cache miss ran in a newly named container from this immutable image ID [14]:

```
sha256:a869cd1dffb8c87afad1bb1302106cb9f5cb580641c7391bb73f4ab077f140d9.
```

Networking is disabled (`-network none`) and the root filesystem is read only. The runtime drops all capabilities, sets `no-new-privileges`, uses non-root UID/GID 65532, and supplies no host bind mount, named volume, Docker socket, secret, or GPU device. Its only explicit mount is a 16 MiB `/tmp` tmpfs with `noexec,nosuid`. It also sets a 32-PID limit, 128 MiB memory with no swap, a 0.5-CPU scheduling quota, and a separate host-enforced wall-clock deadline. The pinned Docker 29.4.1 CLI reference documents these run flags [8]. AST filtering and reduced builtins are defense in depth, not a security boundary.

3.3 Model update

The implementation uses BF16 Transformers/PEFT rather than 4-bit loading. LoRA rank is 16, alpha is 32, dropout is zero, bias is disabled, and `all-linear` targets linear and Conv1D layers except the output layer. This produces 8,798,208 trainable parameters (1.7497% of 502,830,976 total).

The main GRPO configuration is shown in Table 1. The effective optimizer batch is $1 \times 4 = 4$, divisible by the four generations. Reward scaling is disabled, $\beta = 0$ omits a reference-model KL term, the DR-GRPO loss uses the fixed completion-length divisor, and fully truncated completions are masked. Gradient checkpointing and SDPA reduce memory. A callback would stop if more than 80% of groups had zero reward variance for 20 consecutive logs [2, 13, 14].

3.4 Source-committed selection and evaluation

The untouched base, two-step smoke adapter, and 100-step adapter use identical validation prompts, task-level seeds, decoding, and sandbox. Candidate adapters are ranked lexicographically by mean greedy hidden-test fraction and then greedy full-solve rate. The learning gate requires a hidden-fraction point gain of at least 0.05 or one additional solved task, and an aggregate rate of invalid, timeout, runtime, and policy statuses no worse than the base.

The optional 200-step continuation decision was made before any final model generation, score, or outcome was available. Internal final task definitions and hidden cases had already been loaded and

Table 1: Checked main training configuration.

Setting	Value
Steps / seed	100 / 42
Generations per prompt	4
Per-device batch / accumulation	1 / 4
Prompt / completion limit	512 / 256 tokens
Learning rate / warmup	10^{-5} / 0.05
Sampling	temperature 0.9, top- p 0.95
Gradient norm / precision	0.1 / BF16
Loss / KL coefficient	DR-GRPO / $\beta = 0$
LoRA rank / alpha / dropout	16 / 32 / 0
Checkpoint interval	25 steps; retain latest two

structurally checked; HumanEvalPack rows were loaded only in the subsequent evaluation path. The 100-step adapter passed the gate, while mean training rewards in steps 51–75 and 76–100 were 1.5230 and 1.5355. The checkpoint was locked rather than extending the budget opportunistically [14].

Internal evaluation uses one greedy and four sampled candidates per task. “Observed sampled success@4” is the fraction with at least one full solve among exactly four recorded draws, not the unbiased pass@ k estimator discussed by Chen et al. [9]. Reported intervals use 10,000 paired task-level percentile-bootstrap resamples with seed 42.

Only after selection, the same checkpoint is evaluated on HumanEvalFixDocs-Python from HumanEvalPack [10, 11], pinned to dataset revision

9a41762f73a8cb23bb5811b73d5aab164efcf378.

There is one greedy candidate for each of 164 tasks. The prompt conversion and first-block stopping follow the pinned BigCode harness [12]. MiniBug-RL applies a three-second wall-clock deadline to its resource-limited container, stricter than the harness’s 10-second Python limit; the score is therefore not leaderboard-identical.

4 Results

4.1 Training viability

The two-step smoke changed 168 adapter tensors and survived save/reload. Main training reached exactly 100 steps with finite loss 0.017962, changed all 336 trainable tensors, and had adapter ℓ_2 delta 0.281200. Peak allocated GPU memory was 1,849,278,976 bytes (1.72 GiB), below the recorded 8,151 MiB device capacity. The longest zero-reward-variance streak was eight, below the 20-log stop threshold. Four of 400 main-training rollouts were clipped at the completion limit, all by step 31. The structured log records no training, CUDA, or reward-collapse failure; process stderr was not part of that log, so it cannot certify every third-party diagnostic stream [14].

4.2 Validation and locked internal test

Table 2: Internal results. Hidden fraction is a macro-average over tasks.

Metric	Base	Selected adapter
Validation greedy solves	7/12	8/12
Validation greedy hidden fraction	0.7708	0.8333
Final greedy solves	5/12	6/12
Final greedy hidden fraction	0.6083	0.6042
Final observed sampled success@4	10/12	10/12
Final sampled hidden fraction	0.4635	0.6281
Final invalid candidates	19/60	1/60

Validation selected the main adapter with a greedy hidden-fraction difference of +0.0625 (paired percentile-bootstrap 95% interval [-0.1458, 0.2917]) and one additional solved task. The interval crosses zero; the deterministic source-committed gate passed, but this is not statistical significance [14, 15].

On final tasks, greedy full solves differed by one, while mean greedy hidden fraction was essentially unchanged: difference -0.0042, interval [-0.2917, 0.2625]. Both policies passed 30 of 49 raw greedy hidden cases; the small difference in macro means arises because tasks are weighted equally and one task has five cases. Sampled hidden correctness was higher and invalid formatting lower for the selected adapter, although runtime errors rose from four to seven and the selected set added one timeout and one policy violation. Thus the precise structural result is fewer invalid candidates, not an improvement in every failure category [17, 15].

4.3 Frozen external evaluation

Table 3: HumanEvalFixDocs-Python, greedy $n = 1$, 164 tasks, 3-second MiniBug-RL sandbox.

Metric	Base	Selected adapter
Passed	37/164	38/164
Pass@1	0.2256	0.2317
Failed, excluding timeout	124	122
Timeouts	3	4

The paired difference was +0.0061 (paired percentile-bootstrap 95% interval [-0.0183, 0.0366]). Three tasks improved, two regressed, and 159 tied. Timeouts were 3 versus 4. The one-task net gain is compatible with no underlying effect [17, 15].

5 Error analysis and interpretation

The largest observed behavioral difference is format adherence on the internal final protocol: invalid candidates were 19 for base and one for the selected adapter among 60 greedy-plus-sampled candidates per system. This matters operationally because prose, multiple functions, and malformed

source receive host-side penalties without reaching hidden-test execution. The single run does not isolate why sampled partial correctness also differed.

Correctness changes were heterogeneous. Relative to base, greedy internal improvements occurred on Boolean parsing, nearest multiples, wrapped indexing, and pair sums; regressions occurred on Hamming distance, alternating strings, and trace computation. Some selected-adapter outputs were executable but incorrect where base outputs had been invalid, moving observed errors from parsing into runtime behavior. Longer selected-adapter completions coincided with a 2,065.4-second external evaluation versus 1,116.5 seconds for base ($1.85\times$) and one additional timeout; the run did not isolate a causal timing mechanism [17].

Accordingly, the evidence supports the narrow claim that verifiable GRPO completed, changed the adapter tensors, and coincided with fewer invalidly structured internal candidates in this comparison. It does not support a broad claim of improved software engineering. All three paired intervals—validation, internal final, and external—include zero.

6 Experiment history and corrective loop

An initial diagnostic run completed preflight, preparation, baseline, and a two-step smoke check, but its terminal displayed a Trackio conversion warning when TRL emitted undefined clip extrema for an entirely clipped batch. That stderr warning was not persisted in the structured log, and a later-inspected Trackio database was mutable and absent from the artifact manifest; the evidence therefore does not establish that Trackio durably dropped a complete record. The run was stopped manually before main training. The fix subclasses the official callback, removes only **None** fields, and retains zero and negative metrics. Curriculum review at the same boundary found two hidden suites that did not catch their authored bugs and one cross-split structural collision; all were corrected and re-audited before the completed run.

This diagnostic evidence is not pooled with the final experiment because its source, data, and manifest hashes differ. The completed loop was: define a small verifiable domain; state the gate; audit data and isolation; measure the base; prove gradient/save/reload viability; train 100 steps; decide the budget from training plus validation; lock; score each final benchmark once per compared system; export; publish; re-download; and report both positive and neutral outcomes.

7 Threats to validity

Statistical power. The internal sets contain only 12 tasks each and the experiment has one training seed. One solved task moves a pass rate by 8.33 percentage points. Bootstrap intervals quantify paired task uncertainty but not variation across training seeds or curriculum construction.

Selection. Validation is intentionally optimistic because it selects the checkpoint and budget. Only final-test and external results estimate post-selection behavior.

Benchmark exposure. No HumanEvalPack row, model outcome, or score entered training, reward computation, selection, hyperparameter changes, or the budget decision. This data-flow statement does not prove that benchmark concepts were unknown to the authors, and upstream base-model pretraining exposure cannot be ruled out. Both evaluated policies share the latter risk.

Protocol comparability. The external prompt uses the raw HumanEvalFixDocs instruct form

rather than Qwen’s chat template, and the 3-second deadline differs from the BigCode harness. The measured numbers must not be compared directly with stock leaderboard entries.

Security. Containers share the host kernel and are not perfect hostile-code virtual machines. The setup uses no host bind or named-volume mount and no Docker socket, but it does mount an isolated `/tmp` tmpfs. Higher-assurance deployments should add a disposable VM, rootless Docker, user namespaces, seccomp/AppArmor, or a microVM boundary.

Scope. Synthetic pure-function repair does not test repository navigation, dependency management, multi-file patches, tests written by the model, or long-horizon agent behavior.

8 Reproducibility and artifacts

The producing source commit is `7027bc55baecc00fad51cbe2b8f030dca2c91c1e`. Public configuration, curriculum, and manifest SHA-256 hashes are retained with the experiment report [13, 14]. The project required Python 3.12 or newer, but the persisted run environment did not capture the Python patch release. It did record PyTorch 2.14.0+cu130, Transformers 5.16.1, TRL 1.12.0, PEFT 0.20.0, and Docker 29.4.1 on an NVIDIA GeForce RTX 5070 Laptop GPU. Full prompt/completion pairs, reward breakdowns, and per-step metrics were recorded in append-only logs; environment, checkpoint, selection, evaluation, export, and phase state were retained in separate artifacts rather than all being fields of one log.

The public Hugging Face repository is

<https://huggingface.co/BurnyCoder/qwen2.5-coder-0.5b-swe-rl>

at immutable original result revision `5b6e22a4c6c01bec95d10e93a0fc78666eb9c543`. Its root contains the merged model, tokenizer, card, configuration, and measured results; `adapter/` contains the separate LoRA [17]. The original publish phase re-downloaded that exact revision, loaded the root model, and produced a non-empty greedy repair, but did not compare weight hashes. An independent post-publication check later found byte-identical local and downloaded Safetensors: merged SHA-256 `e6b2d3884e72fe8322eaa5fe181ce530593f710b8b4efdcf2012b911d2416a70` and adapter SHA-256 `37f5f71fae87a4fcc5b3df40d325544271d28cf7b55d6a0262c6096d6233b1b2` [16].

9 Conclusion

MiniBug-RL answers the engineering question affirmatively for the recorded system: a complete single-function software-repair RL loop, including verifiable rewards, low-rank updates, split discipline, resource-limited execution, paired evaluation, model export, and public reload verification, completed on an 8 GB GPU at 1.72 GiB measured peak allocated training memory. The scientific result is more restrained. The selected adapter solved one additional task on each selection-held-out benchmark and had substantially fewer malformed internal outputs, but average greedy correctness did not reliably improve and every paired interval includes zero. The appropriate next experiment is a larger structurally diverse training curriculum and multiple seeds, selected without tuning against the current model-scored final sets.

A Exact reproduction commands

```
git clone https://github.com/BurnyCoder/llm-rl-software-engineering.git
cd llm-rl-software-engineering
git checkout 7027bc55baecc00fad51cbe2b8f030dca2c91c1e
uv sync --locked
RUN_ID="$(date -u +%Y%m%dT%H%M%SZ)-minibug-rl-main"
uv run swe-rl all --config configs/local-8gb.toml \
  --run-id "$RUN_ID" --skip-publish
```

Publication is a separate, authorized action after setting `HF_TOKEN` in the ignored mode-600 `.env`:

```
uv run swe-rl publish --config configs/local-8gb.toml \
  --run-id "$RUN_ID"
```

B Evidence map

Machine-readable checked summaries live in `reports/evidence/`. Run-specific hypotheses, failures, fixes, and decisions live in `reports/experiments/`. The operational procedure, stable methodology, threat model, and provenance interpretation live in separate files under `docs/`. Raw logs and model weights remain outside Git and are referenced by run ID and immutable public commits.

References

- [1] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, 2024. <https://arxiv.org/abs/2402.03300>
- [2] Hugging Face. TRL 1.12.0 GRPO Trainer documentation. https://huggingface.co/docs/trl/v1.12.0/en/grpo_trainer
- [3] E. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations*, 2022. <https://arxiv.org/abs/2106.09685>
- [4] Hugging Face. PEFT 0.20.0 LoRA API documentation. https://huggingface.co/docs/peft/v0.20.0/en/package_reference/lora
- [5] B. Hui et al. Qwen2.5-Coder Technical Report. *arXiv:2409.12186*, 2024. <https://arxiv.org/abs/2409.12186>
- [6] Qwen Team. Qwen2.5-Coder-0.5B-Instruct model card at commit `ea3f2471cf1b1f0db85067f1ef93848e38e88c25`. <https://huggingface.co/Qwen/Qwen2.5-Coder-0.5B-Instruct/blob/ea3f2471cf1b1f0db85067f1ef93848e38e88c25/README.md>
- [7] B. V. Tadepalli. DebugArena: Teaching LLMs to Fix Bugs Through Trial and Error. Hugging Face model card at commit `e4be134207727d684b8a7edfbc7934159893617b`, 2026. <https://huggingface.co/BharathVikas/debugarena/blob/e4be134207727d684b8a7edfbc7934159893617b/README.md>

- [8] Docker, Inc. Docker CLI 29.4.1 `run` reference source at commit 055a478ea9010a19d0d4674c0d0e87ade37a4223. <https://github.com/docker/cli/blob/055a478ea9010a19d0d4674c0d0e87ade37a4223/docs/reference/commandline/run.md>
- [9] M. Chen et al. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*, 2021. <https://arxiv.org/abs/2107.03374>
- [10] N. Muennighoff et al. OctoPack: Instruction Tuning Code Large Language Models. *arXiv:2308.07124*, 2023. <https://arxiv.org/abs/2308.07124>
- [11] BigCode Project. HumanEvalPack dataset card at revision 9a41762f73a8cb23bb5811b73d5aab164efcf378. <https://huggingface.co/datasets/bigcode/humanevalpack/blob/9a41762f73a8cb23bb5811b73d5aab164efcf378/README.md>
- [12] BigCode Project. Pinned HumanEvalPack evaluation implementation at commit 8fc5bae6479c4fbbb28c3f8b644f6a15b3f3b5bd. https://github.com/bigcode-project/bigcode-evaluation-harness/blob/8fc5bae6479c4fbbb28c3f8b644f6a15b3f3b5bd/bigcode_eval/tasks/humanevalpack.py
- [13] MiniBug-RL. Producing source commit 7027bc55baecc00fad51cbe2b8f030dca2c91c1e. <https://github.com/BurnyCoder/llm-rl-software-engineering/commit/7027bc55baecc00fad51cbe2b8f030dca2c91c1e>
- [14] MiniBug-RL. Audited measured run summary, schema version 2. `reports/evidence/run-summary.json`, distributed with this paper.
- [15] MiniBug-RL. Exact paired task scores and intervals. `reports/evidence/task-scores.json`, distributed with this paper.
- [16] MiniBug-RL. Amended independent verification record, schema version 2. `reports/evidence/verification.json`, distributed with this paper.
- [17] MiniBug-RL. Immutable published comparison result at revision 5b6e22a4c6c01bec95d10e93a0fc78666eb9c543. <https://huggingface.co/BurnyCoder/qwen2.5-coder-0.5b-swe-rl/blob/5b6e22a4c6c01bec95d10e93a0fc78666eb9c543/results.json>